Assignment No. 10

Title: Implementation of ARIMA Model in Python

Objective:

To implement the ARIMA (Auto-Regressive Integrated Moving Average) model using Python for time series forecasting and analyze how past observations and error terms can be used to predict future values.

Problem Statement:

Implement the ARIMA model using Python on a time series dataset. Load the dataset into a pandas DataFrame and apply the ARIMA technique to forecast future values. Analyze the dataset for stationarity, apply differencing if necessary, and visualize the predicted values along with the original data.

Theory:

Time Series Analysis

Time series analysis is a statistical technique used to analyze data collected over a specific time period. It helps identify patterns such as trends, seasonality, and cyclical movements. Time series models are widely used for forecasting future values based on historical data.

Examples of time series data include stock prices, weather conditions, economic indicators, and sales data.

ARIMA Model

ARIMA stands for **Auto-Regressive Integrated Moving Average**. It is one of the

most widely used models for time series forecasting.

The ARIMA model combines three components:

- **AR (Auto-Regressive)**
- **I (Integrated)**
- **MA (Moving Average)**

It is represented as:

ARIMA(p,d,q)ARIMA(p, d, q)

Where:

- **p** = Order of the Auto-Regressive model
- **d** = Degree of differencing required to make the data stationary
- **q** = Order of the Moving Average model

Components of ARIMA Model

1. Auto-Regressive (AR)

The AR component uses past values of the time series to predict the current value.

Components of ARIMA Model

1. Auto-Regressive (AR)

The AR component uses past values of the time series to predict the current value.

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p}$$

2. Integrated (I)

The Integrated part represents differencing of observations to make the time series stationary.

Differencing formula:

$$Y_t' = Y_t - Y_{t-1}$$

3. Moving Average (MA)

The MA component models the error term as a linear combination of error terms from previous time steps.

$$Y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \ldots + \theta_q \epsilon_{t-q}$$

Applications of ARIMA

1. Stock market forecasting
2. Sales forecasting
3. Weather prediction
4. Economic data analysis
5. Demand forecasting

Implementation using Python

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
```

- **Pandas** is used for data manipulation and analysis.
- **NumPy** is used for numerical operations.
- **Matplotlib** is used for plotting graphs.
- **Statsmodels** provides statistical models for time series analysis.

Step 2: Load the Dataset

```
df = pd.read_csv("data.csv")
```

This loads the dataset into a pandas DataFrame.

Step 3: Convert Date Column into Datetime Format

```
df['Date'] = pd.to_datetime(df['Date'])
```

This converts the Date column into datetime format for time series analysis.

Step 4: Set Date as Index

```
df.set_index('Date', inplace=True)
```

This organizes the dataset based on time.

Step 5: Select the Target Variable

```
data = df['Close']
```

The closing price is selected as the time series variable.

Step 6: Split Dataset into Training and Testing

```
train_size = int(len(data) * 0.8)
train, test = data[:train_size], data[train_size:]
```

- **Training data (80%)** is used to build the model.
- **Testing data (20%)** is used to evaluate predictions.

Step 7: Build the ARIMA Model

```
model = ARIMA(train, order=(1,1,1))
```

This creates an ARIMA model with:

- **p = 1**
- **d = 1**
- **q = 1**

Step 8: Fit the Model

```
model_fit = model.fit()
```

This trains the ARIMA model using the training data.

Step 9: Generate Predictions

```
forecast = model_fit.forecast(steps=len(test))
```
This predicts future values for the test dataset.

Step 10: Plot the Results

```
plt.figure(figsize=(12,6))
plt.plot(train.index, train, label="Train Data")
```

```
plt.plot(test.index, test, label="Actual Data")
plt.plot(test.index, forecast, label="Predicted Data")
plt.legend()
plt.title("ARIMA Model Forecast")
plt.xlabel("Date")
plt.ylabel("Value")
plt.show()
```

This graph shows the **actual values and predicted values**.

Step 11: Evaluate the Model

```
rmse = np.sqrt(mean_squared_error(test, forecast))
print("RMSE:", rmse)
```

RMSE (Root Mean Square Error) measures the difference between predicted and actual values.

Lower RMSE indicates better forecasting accuracy.

Advantages of ARIMA Model

1. Suitable for time series forecasting.
2. Can model different types of time series data.
3. Captures relationships between past values and error terms.
4. Widely used in financial and economic forecasting.

Limitations of ARIMA Model

1. Requires stationary data.
2. Difficult to determine correct parameters (p, d, q).
3. Not suitable for nonlinear patterns.

Conclusion

Thus, the ARIMA model was successfully implemented using Python for time series forecasting. The model combines Auto-Regressive, Integrated, and Moving Average components to analyze historical data and predict future values. The performance of the model was evaluated using RMSE and visualized using graphs.